

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Spring Boot Actuator: In-Depth Analysis and Code Samples



Abhishek Ranjan · [Follow](#)

5 min read · May 13, 2023



40





Spring Boot Actuator is a sub-project of Spring Boot. It provides production-ready features to help you monitor and manage your application. With Actuator, you can add several built-in services to your application without writing any code.

Features of Spring Boot Actuator

Here are some features provided by Spring Boot Actuator:

- **Health check-up:** Spring Boot Actuator provides a /health endpoint to check the health of your application.
- **Metrics gathering:** It gathers various metrics about your application like memory usage, garbage collection, web request tracing, etc.
- **HTTP tracing:** It keeps track of HTTP request-response exchange details.
- **Application info:** It can display application-related information.
- **Shutdown:** It exposes a shutdown endpoint to gracefully shut down the application.
- **Environment information:** It exposes environment variables and configuration properties.

Enabling Spring Boot Actuator

To add the Spring Boot Actuator to your project, you need to add its dependency to your build file.

For Maven:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-actuator</artifactId>  
</dependency>
```

For Gradle:

```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-actuator'  
}
```

Usage

Once you've added the dependency, Spring Boot Actuator's features will automatically become available in a standard Spring Boot application. For instance, you can navigate to <http://localhost:8080/actuator/health> to view your application's health information.

By default, only the 'health' and 'info' endpoints are enabled. If you want to enable all endpoints, you can add the following property to your application.properties file:

```
management.endpoints.web.exposure.include=*
```

If you want to expose only specific endpoints, you can list them:

```
management.endpoints.web.exposure.include=health,info,beans
```

Custom Health Indicator

Spring Boot Actuator allows you to create custom health indicators. Here's an example of a custom health indicator:

```
import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.stereotype.Component;
@Component
public class CustomHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        int errorCode = check(); // perform some specific health check
        if (errorCode != 0) {
            return Health.down()
                .withDetail("Error Code", errorCode).build();
        }
        return Health.up().build();
    }
    public int check() {
        // Our logic to check health
        return 0;
    }
}
```

In the code above, we create a custom health check indicator. If the `check()` method returns anything other than 0, the health status is set to DOWN. Otherwise, it's UP.

Metrics

Spring Boot Actuator provides a `/metrics` endpoint that presents a lot of useful information related to the application. This endpoint shows JVM memory used, system CPU usage, open files, and much more.

Built-In Metrics

Spring Boot Actuator provides a wealth of built-in metrics that can provide invaluable insights into the behavior and performance of your application. These metrics can be accessed through the `/metrics` endpoint. Below, we discuss some of the most common and useful metrics that are provided out of the box:

1. **jvm.memory.used:** This metric provides information about the amount of memory currently used by the JVM.
2. **jvm.memory.committed:** This metric gives you the amount of memory that is guaranteed to be available for use by the JVM.
3. **jvm.memory.max:** This metric shows the maximum amount of memory that the JVM will attempt to use.
4. **jvm.gc.memory.allocated:** This metric provides the total amount of memory allocated by the JVM since it started running.

5. **jvm.gc.memory.promoted:** The total amount of memory currently occupied by survivors — the objects that have survived the garbage collector because they are still being used.
6. **jvm.gc.max.data.size:** The maximum memory size of old generation. This size is either set by the JVM or can be configured using JVM options.
7. **jvm.gc.pause:** Shows the total time that the application has been paused for garbage collection.
8. **jvm.buffer.memory.used:** Amount of memory used by the JVM for the buffer pool.
9. **jvm.buffer.count:** Number of buffers being used by the JVM.
10. **jvm.threads.live:** The current number of live threads in the JVM, including both daemon and non-daemon threads.
11. **jvm.threads.daemon:** The current number of live daemon threads in the JVM.
12. **jvm.threads.peak:** The peak live thread count since the JVM started or peak was reset.
13. **process.uptime:** Displays the uptime of the application.

14. **process.cpu.usage:** The recent cpu usage for the Java Virtual Machine process.
15. **process.start.time:** Displays the start time of the application.
16. **system.cpu.count:** The number of processors available to the Java virtual machine.
17. **system.cpu.usage:** The “recent cpu usage” for the whole system.
18. **http.server.requests:** This is a versatile metric as it provides details on HTTP request handling by the server, including information about the request count, error count, response time, etc.

These are just a few examples of the metrics available. Depending on the libraries used in your application, Spring Boot Actuator may provide additional metrics. For instance, if you use a data source connection pool, there will be metrics for the number of active connections, idle connections,



 Search

 Write



Remember, metrics are a powerful tool for understanding the behavior and performance of your application. It's crucial to understand what each metric means and how it can impact your application's performance.

Securing Actuator Endpoints

Actuator endpoints might expose sensitive information about your application, hence securing these endpoints is crucial. Here's how you can do it with Spring Security:

Add the Spring Security starter to your build file. For Maven:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

For Gradle:

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-security'
}
```

Next, configure Spring Security in your `application.properties`:

```
spring.security.user.name=admin  
spring.security.user.password=admin
```

This will secure all your application endpoints including Actuator endpoints under basic authentication. You can access these endpoints using the username `admin` and password `admin`.

For a more fine-grained control, you can use a `WebSecurityConfigurerAdapter` to secure only the Actuator endpoints:

```
import org.springframework.boot.actuate.autoconfigure.security.servlet.EndpointRequest;  
import org.springframework.boot.autoconfigure.security.servlet.PathRequest;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.security.config.annotation.web.builders.HttpSecurity;  
import org.springframework.security.config.annotation.web.configuration.WebSecurity  
@Configuration  
public class ActuatorSecurity extends WebSecurityConfigurerAdapter {  
    @Override  
    protected void configure(HttpSecurity http) throws Exception {  
        http.requestMatcher(EndpointRequest.toAnyEndpoint()).authorizeRequests()  
            .anyRequest().hasRole("ACTUATOR")  
            .and()  
            .httpBasic();  
    }  
}
```

```
}  
}
```

In this code, only users with the ACTUATOR role can access the Actuator endpoints.

Actuator Endpoints Customization

You can also customize the Actuator endpoints. For example, you can change the path of the endpoints or even the id of the endpoints. This can be done in the `application.properties` file:

```
management.endpoints.web.base-path=/manage  
management.endpoints.web.path-mapping.health=healthcheck
```

With the above configuration, the base path for all endpoints is changed to `/manage` and the `health` endpoint is now accessed with `/healthcheck` instead of `/health`.

In conclusion, Spring Boot Actuator provides a powerful set of features to monitor and manage your Spring Boot application. By using these features, you can ensure that your application is running smoothly and you can quickly identify and fix any problems that might occur.

 [Connect with me on LinkedIn!](#)

I hope you found this article helpful! If you're interested in learning more and staying up-to-date with my latest insights and articles, don't hesitate to connect with me on [LinkedIn](#).

Let's grow our networks, engage in meaningful discussions, and share our experiences in the world of software development and beyond. Looking forward to connecting with you! 😊

[Follow me on LinkedIn](#) 

Spring Boot

Java

Programming

Software Development

Software Engineering